

LegalEase AI - Frontend Architecture & User Interface

Overview

The frontend of **LegalEase AI** is built with **Streamlit** (`frontend/app.py`), styled with custom CSS (`frontend/styles/custom.css`), and modularized into decoupled reusable components (`frontend/components/`). It delivers a responsive, dual-column experience with live feedback, inline editing, and multi-format document downloads.

Component Architecture

```
text
frontend/
├── __init__.py
├── app.py                # Main application entrypoint & session controller
├── components/
│   ├── __init__.py
│   ├── input_form.py    # Form inputs, date pickers, branding uploader
│   ├── preview.py       # Styled HTML preview & inline text editor
│   └── download_buttons.py # Binary stream exports (TXT, DOCX, PDF)
├── utils/
│   ├── __init__.py
│   └── api_client.py    # HTTP client communicating with FastAPI
└── styles/
    └── custom.css       # Custom layout, card, and typography styling
```

1. Input Form Component (`input_form.py`)

The input form guides the user through entering complete contract specifications:

1. **Document Type Selector:** Dropdown supporting 8 contract types:
 - Employment Contract
 - Lease Agreement

- Non-Disclosure Agreement
 - Service Agreement
 - Partnership Agreement
 - Sales Agreement
 - Freelance Contract
 - General Agreement
2. **Parties Involved:** Large text area with pre-populated placeholders for stakeholder names, capacities, corporate status, and addresses.
3. **Key Terms & Conditions:** Large text area supporting semicolon-separated (;) clauses. Each clause is parsed into distinct provisions and an auto-generated Schedule A Terms Table.
4. **Execution & Validity Dates:**
- Agreement Execution Date (`date_input`)
 - Effective Date (`date_input`)
 - Term Start Date (optional `date_input`)
 - Term End Date (optional `date_input`)
5. **Custom Branding / Logo Upload:**
- Supports PNG, JPEG, and WebP files up to 5MB.
 - Validated via Pillow before rendering a live thumbnail preview.
6. **Client-Side Validation:**
- Prevents empty submissions.
 - Validates that End Date cannot be earlier than Start Date.

2. Preview & Inline Editor Component (`preview.py`)

The preview component presents the generated legal draft inside an elegant dark-mode container:

- **Header:** Shows the document title, custom logo (if uploaded) or LegalEase logo, and date.
- **Body:** Renders numbered clauses, styled headers (`h3` , `h4`), bold legal terms, and bulleted lists.
- **Schedule A Terms Table:** Automatically renders a styled HTML table with provision names and specifications.
- **Inline Editor:**
 - Clicking "**Click to Edit Document**" toggles a full-height multiline editor.
 - The user can adjust clauses, add bespoke requirements, or correct typos.

- Clicking **"Save Changes"** updates `st.session_state["edited_document"]` and immediately re-renders both the preview and all export streams.

3. Export & Download Component (`download_buttons.py`)

Provides one-click downloads for three formats:

1.  **Download as .TXT**: Clean UTF-8 plain text with ASCII borders and terms summary.
2.  **Download as .DOCX**: Formatted Microsoft Word document in Times New Roman with headers, footers, logo, and shaded terms table.
3.  **Download as .PDF**: Branded PDF generated via ReportLab with header, footer, page numbering, and signature lines.

Key Rule: All downloads immediately export the **latest edited version** stored in session state. No redundant AI requests are made.

4. Session State Management

LegalEase AI tracks the following session keys in `st.session_state` :

State Variable	Type	Description
<code>generated_document</code>	<code>str</code>	Raw sanitized legal draft returned by Gemini.
<code>edited_document</code>	<code>str</code>	Active version of the draft modified by the user.
<code>document_type</code>	<code>str</code>	Selected contract category.
<code>parsed_terms</code>	<code>List[str]</code>	List of individual terms extracted from semicolon input.
<code>custom_logo_bytes</code>	<code>bytes</code>	Raw binary bytes of user-uploaded branding logo.
<code>is_editing</code>	<code>bool</code>	Flag indicating whether the inline editor is open.
<code>error_message</code>	<code>str</code>	Last error encountered, displayed gracefully.
<code>generation_status</code>	<code>str</code>	Status flag (<code>idle</code> , <code>generating</code> , <code>completed</code>).

5. API Client Integration (`api_client.py`)

All communication with the backend is abstracted into `LegalEaseAPIClient` :

- `check_health()` : Tests backend reachability on `/health` . Displays a green status badge in the sidebar when connected.
- `generate_document(payload)` : Dispatches the structured JSON request with a 60-second timeout. Formats error responses into user-friendly messages.